This is a great feature — let me break this into two parts: first the full biological process with timelines, then a concrete integration plan for MatsyaMitra.

Part 1: The Full Aquaculture Lifecycle (Broodstock → Market)

Stage 1 — Broodstock Conditioning

Duration: 3–6 months (ongoing season preparation)

Broodstock are sexually mature adult fish (typically 3–5 years old) selected for superior genetics — this is where enhanced species like **Amrita Katla** and **Jayanti Rohu** come in. They're kept in dedicated broodstock ponds at controlled densities (100–200 kg/hectare) and fed a high-protein diet (30–35% protein) to bring them to spawning readiness. Water temperature is monitored closely — Indian major carps spawn best at 26–32°C.

What to track: Broodstock count, sex ratio (typically 1:1 male:female), weight of individual fish, feeding logs, and water quality readings.

Stage 2 — Induced Spawning (Hypophysation)

Duration: 12–18 hours

Since Indian major carps don't spawn naturally in static ponds, they require hormone-induced spawning. The standard protocol:

1. **First injection** — Pituitary extract or synthetic hormones (Ovaprim / hCG) given to females; males get a smaller dose
2. **Latency period** — 6–12 hours in circular spawning tanks or hapas (floating net enclosures) with flowing/aerated water
3. **Stripping or natural release** — Eggs and milt are either stripped manually or released naturally into the spawning hapa
4. **Fertilization** — Eggs are mixed with milt and incubated in hatching jars/troughs

What to track: Injection timestamp, hormone type and dose, male/female count used, estimated egg count.

Stage 3 — Hatching (Spawn / Hatchlings)

Duration: 12–20 hours post-fertilization

Fertilized eggs hatch into **spawn** (also called hatchlings or sac fry) in incubation jars or circular troughs with continuous water circulation. At 28°C, hatching typically occurs in 14–16

hours. The sac fry absorb their yolk sac over the next **48–72 hours** — this is the "72 hours" you were told about. They cannot feed externally during this window.

After yolk absorption, they're called **advanced spawn** and are ready for the nursery pond.

Total time from spawning to nursery-ready spawn: ~3–4 days

What to track: Hatch time, estimated hatch rate (%), estimated spawn count (typically counted per lakh — 100,000 units).

Stage 4 — Nursery Pond (Spawn → Fry)

Duration: 21–30 days

Advanced spawn are transferred to prepared nursery ponds (typically 0.05–0.1 hectare, shallow 1–1.5m depth). Stocking density: **3–5 million spawn per hectare**.

Nursery ponds are pre-treated with:

- Lime (to eliminate predators and pathogens)
- Organic manure (to bloom zooplankton — the primary food source for spawn)
- Supplemental feeding starts after Day 5–7 (rice bran + mustard cake at 1:1)

By Day 21–30, spawn grow from **6–8mm to 25–35mm** and are now called **fry**.

Survival rate at this stage: 40–60% for general carps; enhanced species like Jayanti Rohu typically achieve 55–70%.

What to track: Transfer date, stocking count, daily feeding logs, water quality, estimated mortality, and fry count at harvest.

Stage 5 — Rearing Pond (Fry → Fingerling)

Duration: 60–90 days

Fry are moved to larger rearing ponds (0.1–0.2 hectare, 1.5–2m depth) at lower densities: **0.2–0.5 million fry per hectare**. Supplemental feeding continues twice daily. By this stage, fish are competing for food and space, so water quality management becomes critical.

By Day 60–90, fry grow to **8–12 cm length and 5–15g body weight** — these are now **fingerlings**, sellable to grow-out farmers.

What to track: Transfer date, stocking count, feeding schedule, water quality (DO, pH, ammonia), estimated weight gain, and projected fingerling-ready date.

Stage 6 — Fingerling Sales

Two pricing models (as you described):

Model	How it works
Per piece	Count-based; fingerlings are netted and counted in sample batches, then extrapolated
Per kg	Weigh total harvest; divide by average individual weight to estimate count

Average fingerling weight at sale: **5–10g** (so roughly **100–200 fingerlings per kg**). This average weight is critical for the per-kg buyers to estimate their stocking density.

Stage 7 — Grow-Out Pond (Fingerling → Market Fish)

Duration: 8–14 months

Fingerlings are stocked into grow-out ponds (typically 1–5 hectare) at 5,000–10,000 fingerlings/hectare in polyculture combinations (Catla + Rohu + Mrigal or enhanced equivalents). With good feed and water management:

Species	Harvest weight	Time to harvest
Catla / Amrita Katla	1.5–2.5 kg	10–12 months
Rohu / Jayanti Rohu	1–2 kg	10–14 months
Mrigal	0.8–1.5 kg	10–12 months

Enhanced species (Jayanti Rohu, Amrita Katla) typically reach harvest weight 20–30% faster due to improved growth rates and disease resistance.

Part 2: Integrating Hatchery Tracking into MatsyaMitra

Here's how I'd architect this. It adds a parallel "Hatchery" track alongside the existing "Pond/Grow-Out" track, with a handoff transaction between them.

User Roles Involved

Gates Foundation Admin → oversight, cross-hatchery analytics, notifications
Hatchery Operator → manages batches stage by stage

Grow-Out Farmer → receives fingerlings, logs grow-out start (already in app)

Your existing `farmer_category` + a new `user_type` field (`hatchery_operator | farmer | admin`) handles this cleanly.

New Screens to Add (7 screens)

Screen	Route Name	Used by
Hatchery Dashboard	`HatcheryDashboard`	Operator, Admin
Add / Edit Batch	`AddEditBatch`	Operator
Batch Progress Detail	`BatchDetail`	Operator, Admin
Stage Log Entry	`StageLog`	Operator
Fingerling Sales	`FingerlingSales`	Operator
Marketplace Listing	`HatcheryMarketplace`	Farmer (buyer)
Grow-Out Start Log	`GrowOutStart`	Farmer (buyer)

Database Schema Additions (3 new migrations)

Migration 025 — `hatchery_core`

sql
```
CREATE TABLE hatcheries (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  operator_id UUID REFERENCES users(id),
  district    TEXT,
  block       TEXT,
  panchayat   TEXT,
  capacity_kg NUMERIC,          -- broodstock holding capacity
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE hatchery_batches (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  hatchery_id     UUID REFERENCES hatcheries(id),
  species_id      UUID REFERENCES species(id),
  species_variant TEXT,         -- 'Jayanti Rohu', 'Amrita Katla', 'Standard'
```

```
  broodstock_male_count   INT,
  broodstock_female_count INT,
  broodstock_total_kg     NUMERIC,
  spawning_date   TIMESTAMPTZ,   -- Stage 2 start
  current_stage   TEXT CHECK (current_stage IN (
    'broodstock', 'spawning', 'hatching',
    'nursery', 'rearing', 'fingerling_ready', 'sold'
  )),
  estimated_spawn_count   BIGINT,  -- in lakhs
  estimated_fry_count     BIGINT,
  estimated_fingerling_count BIGINT,
  avg_fingerling_weight_g NUMERIC, -- for per-kg ↔ per-piece conversion
  notes           TEXT,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Migration 026 — `hatchery_stage_logs`

sql

```
CREATE TABLE hatchery_stage_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id    UUID REFERENCES hatchery_batches(id),
  stage       TEXT NOT NULL,
  started_at  TIMESTAMPTZ NOT NULL,
  ended_at    TIMESTAMPTZ,        -- null = currently in this stage
  count_at_entry   BIGINT,
  count_at_exit    BIGINT,
  survival_rate_pct NUMERIC,      -- auto-calculated
  water_temp   NUMERIC,
  ph           NUMERIC,
  do_mgl       NUMERIC,           -- dissolved oxygen mg/L
  ammonia_ppm  NUMERIC,
  feed_given_kg NUMERIC,
  observations TEXT,
  logged_by   UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Pre-computed expected stage durations (for countdown & notifications)
CREATE TABLE hatchery_stage_benchmarks (
  stage          TEXT PRIMARY KEY,
  min_days       INT,
  max_days       INT,
  typical_days   INT,
  description    TEXT
);

INSERT INTO hatchery_stage_benchmarks VALUES
```

```
  ('spawning',         0,  1,  0,  'Hormone injection to egg release'),
  ('hatching',         0,  1,  0,  'Fertilization to hatch (12-20 hrs)'),
  ('yolk_absorption',  2,  3,  3,  'Sac fry yolk absorption before nursery'),
  ('nursery',         21, 30, 25,  'Spawn to fry in nursery pond'),
  ('rearing',         60, 90, 75,  'Fry to fingerling in rearing pond');
```

Migration 027 — `fingerling_transactions`

sql

```
CREATE TABLE fingerling_sales (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id        UUID REFERENCES hatchery_batches(id),
  buyer_user_id   UUID REFERENCES users(id),  -- null if walk-in buyer
  buyer_name      TEXT,
  buyer_phone     TEXT,
  buyer_district  TEXT,
  sale_date       TIMESTAMPTZ NOT NULL,
  pricing_model   TEXT CHECK (pricing_model IN ('per_piece', 'per_kg')),
  quantity_pieces INT,
  quantity_kg     NUMERIC,
  avg_weight_g    NUMERIC,      -- used to cross-calculate pieces ↔ kg
  price_per_piece NUMERIC,
  price_per_kg    NUMERIC,
  total_amount    NUMERIC,      -- auto-calculated
  delivery_date   TIMESTAMPTZ,
  notes           TEXT,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE grow_out_start_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sale_id         UUID REFERENCES fingerling_sales(id),
  farmer_user_id  UUID REFERENCES users(id),
  pond_id         UUID REFERENCES ponds(id),   -- links to existing ponds table
  received_date   DATE NOT NULL,
  stocked_date    DATE NOT NULL,               -- when placed in grow-out pond
  fingerling_count INT,
  fingerling_avg_weight_g NUMERIC,
  expected_harvest_date DATE,                  -- auto-calculated from species benchmark
  notes           TEXT,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Notification Logic

This is the key feature Gates Foundation asked for. Here's how to structure it:

```
Batch enters REARING stage
    │
    ├── Day 45 → Push: "Batch [X] at 50% of rearing stage.
    │                   Start identifying buyers."
    │
    ├── Day 60 → Push: "Batch [X] fingerlings may be ready in 15–30 days.
    │                   List on marketplace now."
    │
    ├── Day 72 → Push: "Batch [X] approaching fingerling readiness.
    │                   Confirm average weight via sampling."
    │
    └── Stage marked fingerling_ready → Push to:
          - Hatchery operator: "Batch ready. Record sales."
          - Subscribed buyers in same district: "Fingerlings available near you."
          - Gates Foundation admin: "Hatchery [X] reached fingerling stage."
```

Your existing notification infrastructure (`notificationCenter`, `AsyncStorage` for unread tracking) handles in-app delivery. For push, `expo-notifications` (already likely in your Expo setup) sends the scheduled alerts.

The trigger runs as a **cron job** in your Node backend (you already use `node-cron` for market price refresh):

typescript

```
// runs daily at 6 AM
cron.schedule('0 6 * * *', async () => {
  const activeBatches = await db.query(`
    SELECT b.*, sl.started_at as rearing_started_at
    FROM hatchery_batches b
    JOIN hatchery_stage_logs sl ON sl.batch_id = b.id
    WHERE b.current_stage = 'rearing'
      AND sl.stage = 'rearing' AND sl.ended_at IS NULL
  `);

  for (const batch of activeBatches.rows) {
    const daysInRearing = daysSince(batch.rearing_started_at);
    if ([45, 60, 72].includes(daysInRearing)) {
      await sendHatcheryNotification(batch, daysInRearing);
    }
  }
});
```

The Buyer Handoff Flow (Grow-Out Start)

When a farmer receives fingerlings:

```
Farmer opens app
```

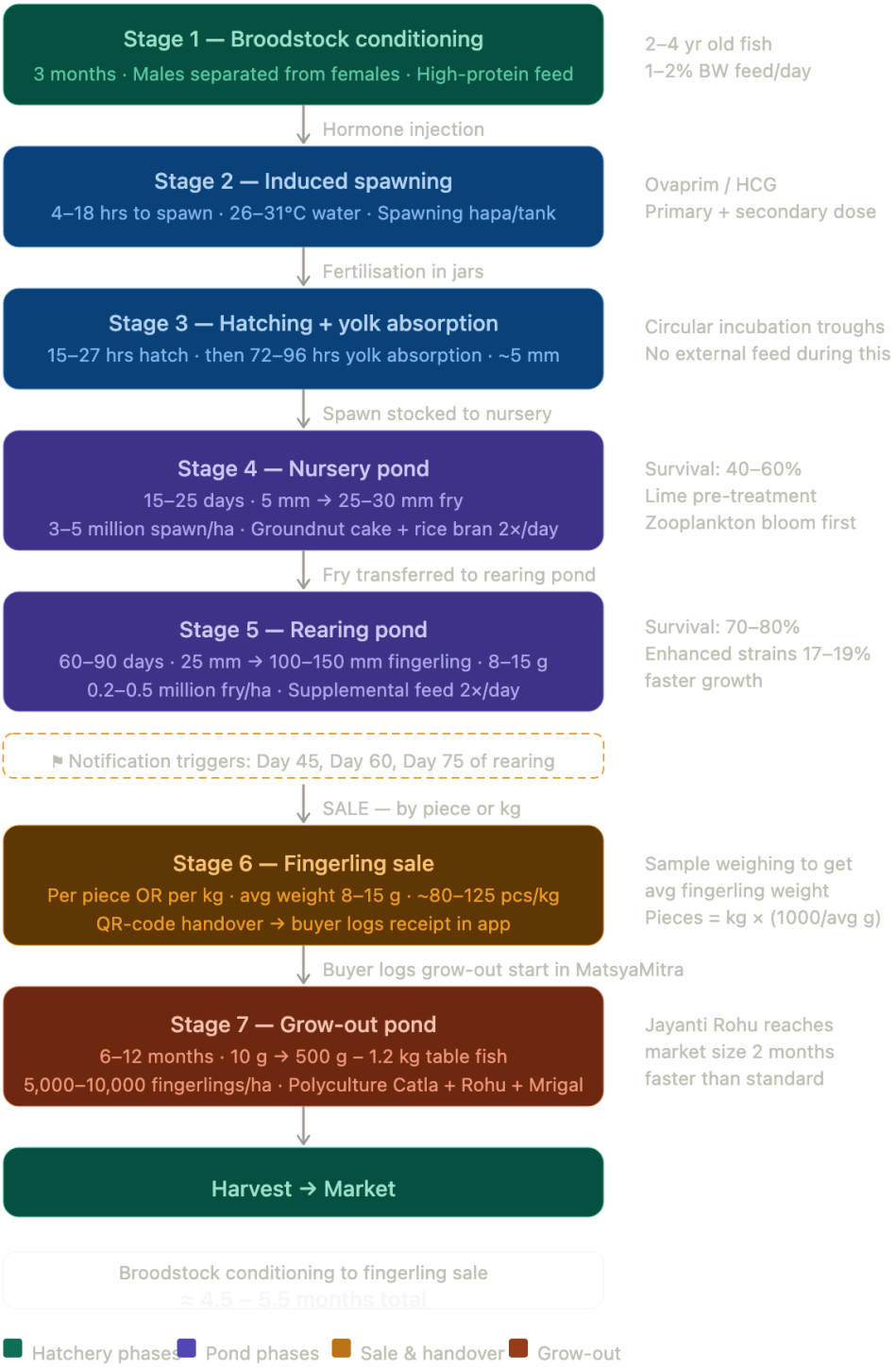
```
│
├── Notification: "You have a pending fingerling delivery from [Hatchery X]"
│   (triggered when hatchery logs the sale with buyer_user_id)
│
├── Farmer taps → GrowOutStart screen
│   - Confirms received_date
│   - Selects which pond (from their existing pond list)
│   - App auto-fills fingerling count from sale record
│   - App auto-calculates expected_harvest_date
│     (stocked_date + species benchmark days)
│
└── On submit → creates grow_out_start_log
        → updates pond record with active batch
        → Harvest Countdown card appears on Home Dashboard
          (your existing bento grid feature!)
```

The last point is important — your **Harvest Countdown** cards on the Home screen already show days-to-harvest per pond. If you link `grow_out_start_log` to the `ponds` table properly, fingerling-sourced ponds automatically appear there with no extra UI work.

Recommended Implementation Order

1. **Migrations 025–027** — get the schema in place
2. **`HatcheryDashboard`** + **`AddEditBatch`** — operators can create batches
3. **`BatchDetail`** + **`StageLog`** — stage progression with manual entries
4. **Cron notification logic** — the core Gates Foundation ask
5. **`FingerlingSales`** — with per-piece / per-kg toggle and auto-conversion
6. **`GrowOutStart`** screen for buyers — with pond picker and harvest date auto-calc
7. **`HatcheryMarketplace`** — discovery for buyers searching by district

This slots cleanly into what you've already built — the location cascade picker handles hatchery geography the same way it handles pond geography, the existing `ponds` table absorbs the grow-out linkage, and the notification bell you already have on the Home screen gets the new alert types with no structural changes.


Stage 1 — Broodstock conditioning
3 months · Males separated from females · High-protein feed
2–4 yr old fish
1–2% BW feed/day
Hormone injection
Stage 2 — Induced spawning
4–18 hrs to spawn · 26–31°C water · Spawning hapa/tank
Ovaprim / HCG
Primary + secondary dose
Fertilisation in jars
Stage 3 — Hatching + yolk absorption
15–27 hrs hatch · then 72–96 hrs yolk absorption · ~5 mm
Circular incubation troughs
No external feed during this
Spawn stocked to nursery
Stage 4 — Nursery pond
15–25 days · 5 mm → 25–30 mm fry
3–5 million spawn/ha · Groundnut cake + rice bran 2×/day
Survival: 40–60%
Lime pre-treatment
Zooplankton bloom first
Fry transferred to rearing pond
Stage 5 — Rearing pond
60–90 days · 25 mm → 100–150 mm fingerling · 8–15 g
0.2–0.5 million fry/ha · Supplemental feed 2×/day
Survival: 70–80%
Enhanced strains 17–19%
faster growth
⚑ Notification triggers: Day 45, Day 60, Day 75 of rearing
SALE — by piece or kg
Stage 6 — Fingerling sale
Per piece OR per kg · avg weight 8–15 g · ~80–125 pcs/kg
QR-code handover → buyer logs receipt in app
Sample weighing to get
avg fingerling weight
Pieces = kg × (1000/avg g)
Buyer logs grow-out start in MatsyaMitra
Stage 7 — Grow-out pond
6–12 months · 10 g → 500 g – 1.2 kg table fish
5,000–10,000 fingerlings/ha · Polyculture Catla + Rohu + Mrigal
Jayanti Rohu reaches
market size 2 months
faster than standard
Harvest → Market
Broodstock conditioning to fingerling sale
≈ 4.5 – 5.5 months total
Hatchery phases
Pond phases
Sale & handover
Grow-out

Spawning → Fingerling	Notification trigger	Pieces per kg	Grow-out harvest
99 days	Day 89	~83	10 months

Yolk absorption (days):	3	Nursery (days):	21	Rearing (days):	75
Avg weight (g):	12				

☐ Sell by piece count ☐ Sell by weight (kg)

Average fingerling weight (g)	Number of pieces
12	10000

Price per piece (₹)	Price per kg (₹)
2.5	250

Estimated pieces	Total weight	Total amount
10,000	120.00 kg	₹25,000

Pieces per kg: **83**

Formula: pieces = kg × (1000 ÷ avg_g)

QR transaction payload preview


```
{
  "species": "Jayanti Rohu",
  "avgWeightG": 12,
  "pricingModel": "PIECES",
  "pieces": 10000,
  "totalKg": 120,
  "totalAmount": 25000,
  "soldDate": "2026-05-26"
}
```

MatsyaMitra — Hatchery Tracking Feature: Full Implementation Guide

For: Gates Foundation Integration
Version: 1.0 — May 2026
Scope: Biological reference + database migrations + backend routes + mobile screens + notifications + sales calculator

---

Part 1 — Validated Biological Timeline (Corrected & Combined)

Stage	Duration	Size transition	Key note
Broodstock conditioning	3 months	Mature adults 2–4 yr	Male/female separated, high-protein feed
Induced spawning	4–18 hrs to release	—	Ovaprim/HCG primary + secondary dose
Hatching	15–27 hrs post-fertilisation	Larval hatchling ~2 mm	26–31°C required
Yolk-sac absorption	72–96 hrs (3–4 days)	2 mm → 5–6 mm spawn	No external feed; starvation risk after
Nursery pond	15–25 days	5–6 mm → 25–30 mm fry	3–5M spawn/ha; zooplankton + rice bran feed
Rearing pond	60–90 days	25 mm → 100–150 mm fingerling	8–15 g; survival 70–80%
Total hatch→fingerling	**≈ 99 days typical**		Notification trigger: Day 89 (10 days before)
Grow-out pond	6–12 months	10 g → 500 g–1.2 kg	Polyculture; Jayanti Rohu 2 months faster

Correction note: The "72 hours" is the yolk-sac absorption window AFTER hatching — not the spawning duration. Spawning itself occurs within 4–6 hours of the second hormone dose.

Enhanced strains advantage (Jayanti Rohu, Amrita Katla):

- 17–19% higher growth rate per generation

- Reach market weight ~2 months faster than standard stock
- Innate resistance to Aeromoniasis (Aeromonas hydrophila)

Part 2 — New User Roles

Add a `user_type` field to the existing `users` table:

```
-- Migration 025_user_type.sql
ALTER TABLE users ADD COLUMN IF NOT EXISTS user_type TEXT
  CHECK (user_type IN ('farmer', 'hatchery_operator', 'admin'))
  DEFAULT 'farmer';

ALTER TABLE users ADD COLUMN IF NOT EXISTS affiliated_hatchery_id UUID;
```

Role	Access
`farmer`	Existing screens + Grow-out start log + Marketplace browse
`hatchery_operator`	Hatchery dashboard + Batch management + Sales recording
`admin` (Gates Foundation)	Cross-hatchery analytics + All notifications

---

Part 3 — Database Migrations

Migration 025 — `hatchery_core`

```
-- 025_hatchery_core.sql

CREATE TABLE IF NOT EXISTS hatcheries (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name            TEXT NOT NULL,
  operator_id     UUID REFERENCES users(id),
  district        TEXT,
  block           TEXT,
  panchayat       TEXT,
  location_code   TEXT,              -- BR-PATNA-SADAR format
  capacity_kg     NUMERIC,
  license_number  TEXT,
  is_active       BOOLEAN DEFAULT true,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE IF NOT EXISTS hatchery_batches (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  hatchery_id           UUID REFERENCES hatcheries(id) ON DELETE CASCADE,
  batch_number          TEXT NOT NULL,          -- e.g. HB-2026-001
  species_id            UUID REFERENCES species(id),
  species_variant       TEXT,                   -- 'Jayanti Rohu' | 'Amrita Katla' | 'Standard'
  broodstock_male_count   INT,
  broodstock_female_count INT,
  broodstock_total_kg     NUMERIC,
  spawning_date         TIMESTAMPTZ,            -- Stage 2 start (hormone injection)
  hatching_date         TIMESTAMPTZ,            -- Stage 3 start
  nursery_stock_date    TIMESTAMPTZ,            -- Stage 4 start
  rearing_stock_date    TIMESTAMPTZ,            -- Stage 5 start
  estimated_fingerling_date DATE,               -- auto-calc: rearing_stock_date + 75 days
  current_stage         TEXT CHECK (current_stage IN (
    'broodstock','spawning','hatching','yolk_absorption',
    'nursery','rearing','fingerling_ready','sold','cancelled'
  )) DEFAULT 'broodstock',
  estimated_spawn_count   BIGINT,               -- in individual units
  estimated_fry_count     BIGINT,
  estimated_fingerling_count BIGINT,
  avg_fingerling_weight_g NUMERIC DEFAULT 12,   -- default 12g; updated after sampling
  notes                 TEXT,
  created_by            UUID REFERENCES users(id),
  created_at            TIMESTAMPTZ DEFAULT NOW(),
  updated_at            TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS hatchery_stage_benchmarks (
  stage         TEXT PRIMARY KEY,
  min_days      INT,
  max_days      INT,
  typical_days  INT,
  description   TEXT
);

INSERT INTO hatchery_stage_benchmarks VALUES
  ('spawning',        0,  1,  0,  'Hormone injection to egg release (4-18 hours)'),
  ('hatching',        1,  2,  1,  'Fertilisation to hatch (15-27 hours)'),
  ('yolk_absorption', 3,  4,  3,  'Sac fry yolk absorption — no external feed'),
  ('nursery',        15, 25, 21,  'Spawn to fry in nursery pond'),
  ('rearing',        60, 90, 75,  'Fry to fingerling in rearing pond')
ON CONFLICT (stage) DO UPDATE SET
  typical_days = EXCLUDED.typical_days,
  description  = EXCLUDED.description;
```

Migration 026 — `hatchery_stage_logs`

```
-- 026_hatchery_stage_logs.sql

CREATE TABLE IF NOT EXISTS hatchery_stage_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id        UUID REFERENCES hatchery_batches(id) ON DELETE CASCADE,
  stage           TEXT NOT NULL,
  started_at      TIMESTAMPTZ NOT NULL,
  ended_at        TIMESTAMPTZ,                -- null = currently in this stage
  count_at_entry  BIGINT,
  count_at_exit   BIGINT,
  survival_rate_pct NUMERIC GENERATED ALWAYS AS (
    CASE WHEN count_at_entry > 0 AND count_at_exit IS NOT NULL
    THEN ROUND((count_at_exit::NUMERIC / count_at_entry) * 100, 1)
    ELSE NULL END
  ) STORED,
  water_temp_c    NUMERIC,
  ph              NUMERIC,
  do_mgl          NUMERIC,
  ammonia_ppm     NUMERIC,
  nitrite_ppm     NUMERIC,
  feed_given_kg   NUMERIC,
  observations    TEXT,
  logged_by       UUID REFERENCES users(id),
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Migration 027 — `fingerling_transactions`

```
-- 027_fingerling_transactions.sql

CREATE TABLE IF NOT EXISTS fingerling_sales (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id        UUID REFERENCES hatchery_batches(id),
  seller_hatchery_id  UUID REFERENCES hatcheries(id),
  buyer_user_id   UUID REFERENCES users(id),      -- null if walk-in buyer
  buyer_name      TEXT,
  buyer_phone     TEXT,
  buyer_district  TEXT,
  sale_date       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  pricing_model   TEXT CHECK (pricing_model IN ('per_piece','per_kg')) NOT NULL,
  quantity_pieces INT,
  quantity_kg     NUMERIC,
  avg_weight_g    NUMERIC NOT NULL,           -- REQUIRED for conversion
  price_per_piece NUMERIC,
  price_per_kg    NUMERIC,
```

```
  total_amount        NUMERIC,
  delivery_date       TIMESTAMPTZ,
  transaction_ref     TEXT UNIQUE,                  -- for QR code lookup
  status              TEXT CHECK (status IN ('pending','delivered','cancelled'))
                      DEFAULT 'pending',
  notes               TEXT,
  created_at          TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS grow_out_start_logs (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sale_id             UUID REFERENCES fingerling_sales(id),
  farmer_user_id      UUID REFERENCES users(id),
  pond_id             UUID REFERENCES ponds(id),  -- links to existing ponds table
  received_date       DATE NOT NULL,
  stocked_date        DATE NOT NULL,              -- when placed in grow-out pond
  fingerling_count    INT,
  fingerling_avg_weight_g NUMERIC,
  expected_harvest_date   DATE,                   -- stocked_date + species benchmark
  actual_harvest_date     DATE,
  harvest_weight_kg       NUMERIC,
  notes               TEXT,
  created_at          TIMESTAMPTZ DEFAULT NOW()
);
```

Part 4 — Backend API Routes

Add these files to `backend/src/routes/`:

`hatcheries.ts`

```
// backend/src/routes/hatcheries.ts
import { Router } from 'express';
import { authenticateToken } from '../middleware/auth';
import pool from '../db';

const router = Router();

// GET /api/v1/hatcheries — list (admin sees all, operator sees own)
router.get('/', authenticateToken, async (req, res) => {
  const { user } = req as any;
  const query = user.user_type === 'admin'
    ? 'SELECT * FROM hatcheries WHERE is_active = true ORDER BY created_at DESC'
    : 'SELECT * FROM hatcheries WHERE operator_id = $1 AND is_active = true';
  const params = user.user_type === 'admin' ? [] : [user.id];
```

```
  const { rows } = await pool.query(query, params);
  res.json(rows);
});

// GET /api/v1/hatcheries/:id/batches
router.get('/:id/batches', authenticateToken, async (req, res) => {
  const { rows } = await pool.query(
    `SELECT b.*, s.name as species_name
     FROM hatchery_batches b
     LEFT JOIN species s ON b.species_id = s.id
     WHERE b.hatchery_id = $1
     ORDER BY b.created_at DESC`,
    [req.params.id]
  );
  res.json(rows);
});

// POST /api/v1/hatcheries/:id/batches — create batch
router.post('/:id/batches', authenticateToken, async (req, res) => {
  const {
    batch_number, species_id, species_variant,
    broodstock_male_count, broodstock_female_count,
    broodstock_total_kg, spawning_date, notes
  } = req.body;

  // Auto-calculate estimated fingerling date: spawning + 1 (hatch) + 3 (yolk) + 21 (nursery) +
75 (rearing)
  const spawnDate = new Date(spawning_date);
  const estDate = new Date(spawnDate);
  estDate.setDate(estDate.getDate() + 100); // 1+3+21+75

  const { rows } = await pool.query(
    `INSERT INTO hatchery_batches
     (hatchery_id, batch_number, species_id, species_variant,
      broodstock_male_count, broodstock_female_count, broodstock_total_kg,
      spawning_date, estimated_fingerling_date, current_stage, notes, created_by)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,'spawning',$10,$11)
     RETURNING *`,
    [req.params.id, batch_number, species_id, species_variant,
     broodstock_male_count, broodstock_female_count, broodstock_total_kg,
     spawning_date, estDate.toISOString().split('T')[0], notes, (req as any).user.id]
  );
  res.status(201).json(rows[0]);
});

// PATCH /api/v1/hatcheries/batches/:batchId/stage — advance stage
router.patch('/batches/:batchId/stage', authenticateToken, async (req, res) => {
  const { new_stage, count_at_entry, observations } = req.body;
```

```
  const { batchId } = req.params;

  await pool.query(
    `UPDATE hatchery_stage_logs SET ended_at = NOW()
     WHERE batch_id = $1 AND ended_at IS NULL`,
    [batchId]
  );
  await pool.query(
    `INSERT INTO hatchery_stage_logs (batch_id, stage, started_at, count_at_entry,
observations, logged_by)
     VALUES ($1, $2, NOW(), $3, $4, $5)`,
    [batchId, new_stage, count_at_entry, observations, (req as any).user.id]
  );
  const { rows } = await pool.query(
    `UPDATE hatchery_batches SET current_stage = $1, updated_at = NOW()
     WHERE id = $2 RETURNING *`,
    [new_stage, batchId]
  );
  res.json(rows[0]);
});

// POST /api/v1/hatcheries/sales — record fingerling sale
router.post('/sales', authenticateToken, async (req, res) => {
  const {
    batch_id, buyer_user_id, buyer_name, buyer_phone, buyer_district,
    pricing_model, quantity_pieces, quantity_kg, avg_weight_g,
    price_per_piece, price_per_kg, total_amount, delivery_date, notes
  } = req.body;

  const txRef = `TXN-${Date.now()}-${Math.random().toString(36).slice(2,7).toUpperCase()}`;
  const hatchery = await pool.query(
    'SELECT hatchery_id FROM hatchery_batches WHERE id = $1', [batch_id]
  );

  const { rows } = await pool.query(
    `INSERT INTO fingerling_sales
     (batch_id, seller_hatchery_id, buyer_user_id, buyer_name, buyer_phone,
      buyer_district, pricing_model, quantity_pieces, quantity_kg, avg_weight_g,
      price_per_piece, price_per_kg, total_amount, delivery_date,
      transaction_ref, notes)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16)
     RETURNING *`,
    [batch_id, hatchery.rows[0]?.hatchery_id, buyer_user_id, buyer_name,
     buyer_phone, buyer_district, pricing_model, quantity_pieces, quantity_kg,
     avg_weight_g, price_per_piece, price_per_kg, total_amount, delivery_date,
     txRef, notes]
  );
  await pool.query(
```

```
    `UPDATE hatchery_batches SET current_stage = 'sold', updated_at = NOW() WHERE id
= $1`,
    [batch_id]
  );
  res.status(201).json(rows[0]);
});

// GET /api/v1/hatcheries/sales/:txRef — buyer looks up transaction by QR code
router.get('/sales/:txRef', authenticateToken, async (req, res) => {
  const { rows } = await pool.query(
    `SELECT fs.*, b.species_variant, b.batch_number, s.name as species_name
     FROM fingerling_sales fs
     JOIN hatchery_batches b ON fs.batch_id = b.id
     JOIN species s ON b.species_id = s.id
     WHERE fs.transaction_ref = $1`,
    [req.params.txRef]
  );
  if (!rows.length) return res.status(404).json({ error: 'Transaction not found' });
  res.json(rows[0]);
});

// POST /api/v1/hatcheries/grow-out — buyer logs stocking
router.post('/grow-out', authenticateToken, async (req, res) => {
  const {
    sale_id, pond_id, received_date, stocked_date,
    fingerling_count, fingerling_avg_weight_g, notes
  } = req.body;

  const sale = await pool.query(
    `SELECT b.species_id FROM fingerling_sales fs
     JOIN hatchery_batches b ON fs.batch_id = b.id WHERE fs.id = $1`,
    [sale_id]
  );
  // Grow-out: 10 months typical for IMC
  const stockedDt = new Date(stocked_date);
  stockedDt.setMonth(stockedDt.getMonth() + 10);
  const expectedHarvest = stockedDt.toISOString().split('T')[0];

  const { rows } = await pool.query(
    `INSERT INTO grow_out_start_logs
     (sale_id, farmer_user_id, pond_id, received_date, stocked_date,
      fingerling_count, fingerling_avg_weight_g, expected_harvest_date, notes)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9) RETURNING *`,
    [sale_id, (req as any).user.id, pond_id, received_date, stocked_date,
     fingerling_count, fingerling_avg_weight_g, expectedHarvest, notes]
  );
  // Update pond with harvest date so Home dashboard Harvest Countdown works
  await pool.query(
```

```
    `UPDATE ponds SET expected_harvest_date = $1 WHERE id = $2`,
    [expectedHarvest, pond_id]
  );
  res.status(201).json(rows[0]);
});

export default router;
```

Register in `backend/src/index.ts`:

```
import hatcheriesRouter from './routes/hatcheries';
app.use('/api/v1/hatcheries', hatcheriesRouter);
```

Part 5 — Notification Cron Job

```
// backend/src/cron/hatcheryNotifications.ts
import cron from 'node-cron';
import pool from '../db';

function daysBetween(date1: Date, date2: Date): number {
  return Math.floor((date2.getTime() - date1.getTime()) / (1000 * 60 * 60 * 24));
}

export function startHatcheryCron() {
  // Run every day at 6:00 AM IST
  cron.schedule('0 0 30 6 * * *', async () => {
    const now = new Date();

    // Find all batches currently in rearing stage
    const { rows: batches } = await pool.query(`
      SELECT b.*, sl.started_at as rearing_started,
             h.name as hatchery_name,
             u.phone as operator_phone,
             u.id as operator_id
      FROM hatchery_batches b
      JOIN hatcheries h ON b.hatchery_id = h.id
      JOIN users u ON h.operator_id = u.id
      JOIN hatchery_stage_logs sl ON sl.batch_id = b.id
      WHERE b.current_stage = 'rearing'
        AND sl.stage = 'rearing'
        AND sl.ended_at IS NULL
    `);

    for (const batch of batches) {
      const daysInRearing = daysBetween(new Date(batch.rearing_started), now);
```

```
      let message: string | null = null;

      if (daysInRearing === 45) {
        message = `Batch ${batch.batch_number} is at Day 45 of rearing (50%). Begin
identifying buyers now.`;
      } else if (daysInRearing === 60) {
        message = `Batch ${batch.batch_number} fingerlings ready in ~15 days. List on
MatsyaMitra marketplace.`;
      } else if (daysInRearing === 75 - 10) {
        message = `URGENT: Batch ${batch.batch_number} is 10 days from fingerling stage.
Confirm average weight via sampling and contact buyers immediately.`;
      } else if (batch.estimated_fingerling_date) {
        const daysToReady = daysBetween(now, new Date(batch.estimated_fingerling_date));
        if (daysToReady === 0) {
          message = `Batch ${batch.batch_number} has reached fingerling stage. Record your
sale now.`;
        }
      }

      if (message) {
        await pool.query(
          `INSERT INTO notifications (user_id, title, body, type, reference_id, reference_type)
           VALUES ($1, $2, $3, 'hatchery_alert', $4, 'hatchery_batch')`,
          [batch.operator_id, 'Hatchery Alert', message, batch.id]
        );
        // Also notify Gates Foundation admin users
        await pool.query(
          `INSERT INTO notifications (user_id, title, body, type, reference_id, reference_type)
           SELECT id, $1, $2, 'hatchery_alert', $3, 'hatchery_batch'
           FROM users WHERE user_type = 'admin'`,
          ['Hatchery Alert — ' + batch.hatchery_name, message, batch.id]
        );
      }
    }
  });
}
```

Add to `backend/src/index.ts`:

```
import { startHatcheryCron } from './cron/hatcheryNotifications';
startHatcheryCron();
```

Part 6 — Mobile Utility Files

src/utils/seedSalesCalculator.ts

```
interface SaleInput {
  species: string;
  averageWeightGrams: number;
  saleMethod: 'PIECES' | 'WEIGHT';
  quantityInput: number;
}

interface SaleResult {
  totalEstimatedPieces: number;
  totalWeightKg: number;
  piecesPerKg: number;
  transactionPayload: object;
}

export function calculateFingerlingSale(input: SaleInput): SaleResult {
  const { species, averageWeightGrams, saleMethod, quantityInput } = input;

  if (averageWeightGrams <= 0) {
    throw new Error('Average fingerling weight must be greater than 0');
  }

  const piecesPerKg = Math.round(1000 / averageWeightGrams);
  let totalEstimatedPieces: number;
  let totalWeightKg: number;

  if (saleMethod === 'PIECES') {
    totalEstimatedPieces = Math.round(quantityInput);
    totalWeightKg = parseFloat(((totalEstimatedPieces * averageWeightGrams) /
1000).toFixed(2));
  } else {
    totalWeightKg = quantityInput;
    totalEstimatedPieces = Math.round(totalWeightKg * (1000 / averageWeightGrams));
  }

  return {
    totalEstimatedPieces,
    totalWeightKg,
    piecesPerKg,
    transactionPayload: {
      species,
      avgWeightG: averageWeightGrams,
      pricingModel: saleMethod,
      pieces: totalEstimatedPieces,
      totalKg: totalWeightKg,
      soldDate: new Date().toISOString().split('T')[0],
    }
```

```
  };
}
```

`src/services/hatcheryNotifications.ts` (mobile — local push)

```
import * as Notifications from 'expo-notifications';

const DAYS_HATCH      = 1;
const DAYS_YOLK       = 3;
const DAYS_NURSERY    = 21;
const DAYS_REARING    = 75;
const TOTAL_DAYS      = DAYS_HATCH + DAYS_YOLK + DAYS_NURSERY +
DAYS_REARING; // 100

export function calculateFingerlingMaturityDate(spawningDate: Date): Date {
  const d = new Date(spawningDate);
  d.setDate(d.getDate() + TOTAL_DAYS);
  return d;
}

export async function scheduleHatcheryAlerts(batchNo: string, spawningTimestamp:
number) {
  const spawning  = new Date(spawningTimestamp);
  const maturity  = calculateFingerlingMaturityDate(spawning);
  const rearingStart = new Date(spawning);
  rearingStart.setDate(rearingStart.getDate() + DAYS_HATCH + DAYS_YOLK +
DAYS_NURSERY);

  const alerts = [
    {
      daysFromRearing: 45,
      title: 'Fingerling Alert — 50% through rearing',
      body:  `Batch ${batchNo}: Start identifying buyers. ~30 days remaining.`
    },
    {
      daysFromRearing: 60,
      title: 'Fingerling Alert — 80% through rearing',
      body:  `Batch ${batchNo}: List fingerlings on marketplace now. ~15 days remaining.`
    },
    {
      daysFromRearing: DAYS_REARING - 10,
      title: '🚨 Fingerlings ready in 10 days',
      body:  `Batch ${batchNo}: Sample-weigh fingerlings and finalise buyers immediately.`
    },
  ];

  for (const alert of alerts) {
```

```
    const triggerDate = new Date(rearingStart);
    triggerDate.setDate(triggerDate.getDate() + alert.daysFromRearing);
    if (triggerDate.getTime() <= Date.now()) continue;

    await Notifications.scheduleNotificationAsync({
      content: {
        title: alert.title,
        body:  alert.body,
        data:  { batchNo, targetScreen: 'BatchDetail' },
      },
      trigger: { date: triggerDate } as any,
    });
  }
}
```

Part 7 — New Screen Checklist (7 screens)

Screen 1: `HatcheryDashboard`

- Operator/admin landing page
- Bento grid: active batches count, batches in rearing, fingerling-ready batches, total sales this season
- List of batches sorted by urgency (days to fingerling_ready ascending)
- FAB: "+ New Batch"

Screen 2: `AddEditBatch`

- Fields: batch number (auto-generate), species picker, variant (Jayanti Rohu / Amrita Katla / Standard), broodstock counts M+F, total kg, spawning date
- On save: auto-calculate `estimated_fingerling_date`, schedule local push notifications via `scheduleHatcheryAlerts()`
- Shows estimated fingerling date preview before save

Screen 3: `BatchDetail`

- Timeline progress bar across 5 stages (hatching → yolk → nursery → rearing → fingerling_ready)
- Days elapsed in current stage vs benchmark
- Countdown: "X days to fingerling stage"
- "Advance Stage" button → opens `StageLog` modal
- Water quality log entry shortcut (reuses existing WaterQuality component)
- "Record Sale" button appears only when `current_stage === 'fingerling_ready'`

Screen 4: `StageLog` (modal/bottom sheet)

- Stage confirmation (read-only — shows next stage name)
- Count at entry field (estimated number of fish entering new stage)
- Water quality quick entry (DO, pH, temp)
- Observations text field
- Survival rate auto-shown if previous count available

Screen 5: `FingerlingSales`

- Toggle: Sell by Piece / Sell by Weight
- Average weight input (g) — critical field, shown prominently
- Quantity input changes label based on mode
- Live calculation panel: pieces ↔ kg ↔ pieces-per-kg
- Buyer section: search existing users by phone or enter manually
- Price fields: per-piece and per-kg
- Preview of QR transaction payload
- "Generate Sale Record" → posts to API → navigates to success screen with QR

Screen 6: `HatcheryMarketplace` (Farmers/buyers)

- District-filtered list of available fingerling batches
- Each card: species, variant, hatchery name, estimated pieces available, price range, pickup district
- "Request Purchase" button → opens chat/phone link
- Filter by species, variant, district

Screen 7: `GrowOutStart`

- Triggered by notification OR accessed from "Log New Stocking" button
- Transaction reference input (manual) OR QR scanner
- Auto-fills: species, variant, fingerling count, avg weight (from sale record)
- Pond picker (existing pond list from WatermelonDB)
- Received date + Stocked date pickers
- Expected harvest date shown (auto-calculated, 10 months from stocked date)
- On confirm: creates `grow_out_start_log`, updates pond record → Harvest Countdown card auto-appears on Home dashboard

---

Part 8 — WatermelonDB Schema (Mobile Offline)

```
// Add to src/database/schema.js — increment version to 5

tableSchema({
  name: 'hatchery_batches',
  columns: [
```

```
    { name: 'server_id',              type: 'string', isOptional: true },
    { name: 'hatchery_id',            type: 'string' },
    { name: 'batch_number',           type: 'string' },
    { name: 'species_name',           type: 'string' },
    { name: 'species_variant',        type: 'string' },
    { name: 'current_stage',          type: 'string' },
    { name: 'spawning_date',          type: 'number' },       // timestamp
    { name: 'estimated_fingerling_date',type: 'number' },
    { name: 'estimated_fingerling_count',type:'number', isOptional: true },
    { name: 'avg_fingerling_weight_g',  type: 'number' },
    { name: 'notes',                  type: 'string', isOptional: true },
    { name: 'synced',                 type: 'boolean' },
    { name: 'created_at',             type: 'number' },
  ]
}),

tableSchema({
  name: 'grow_out_stockings',
  columns: [
    { name: 'server_id',              type: 'string', isOptional: true },
    { name: 'pond_id',                type: 'string' },
    { name: 'transaction_ref',        type: 'string' },
    { name: 'species_name',           type: 'string' },
    { name: 'species_variant',        type: 'string' },
    { name: 'fingerling_count',       type: 'number' },
    { name: 'fingerling_avg_weight_g',  type: 'number' },
    { name: 'stocked_date',           type: 'number' },
    { name: 'expected_harvest_date',    type: 'number' },
    { name: 'notes',                  type: 'string', isOptional: true },
    { name: 'synced',                 type: 'boolean' },
    { name: 'created_at',             type: 'number' },
  ]
}),
```

Part 9 — Navigation Integration

Add to your bottom tab navigator or drawer:

```
// Add to src/navigation (role-gated)
// For hatchery_operator users:
<Stack.Screen name="HatcheryDashboard"  component={HatcheryDashboardScreen} />
<Stack.Screen name="AddEditBatch"       component={AddEditBatchScreen} />
<Stack.Screen name="BatchDetail"        component={BatchDetailScreen} />
<Stack.Screen name="FingerlingSales"    component={FingerlingSalesScreen} />
```

```
// For all users:
<Stack.Screen name="HatcheryMarketplace" component={HatcheryMarketplaceScreen} />
<Stack.Screen name="GrowOutStart"        component={GrowOutStartScreen} />
```

Role check helper:

```
// src/utils/roleGuard.ts
import { useAuth } from '../AuthContext';

export function useIsHatcheryOperator() {
  const { user } = useAuth();
  return user?.user_type === 'hatchery_operator' || user?.user_type === 'admin';
}
```

Part 10 — Implementation Order

Priority	Task	Effort
1	Migrations 025–027 + register routes	1 day
2	`HatcheryDashboard` + `AddEditBatch`	2 days
3	`BatchDetail` + `StageLog` modal	2 days
4	Cron notification job (server) + local push (mobile)	1 day
5	`FingerlingSales` with dual-mode calculator	1.5 days
6	`GrowOutStart` + pond linkage + Harvest Countdown auto-update	1.5 days
7	`HatcheryMarketplace`	1 day
8	WatermelonDB offline schema + sync	1.5 days
Total		**~11.5 dev days**

Appendix — Key API Endpoints Summary

Method	Route	Used by
GET	`/api/v1/hatcheries`	Operator/admin — list hatcheries

GET	`/api/v1/hatcheries/:id/batches`	Operator — list batches
POST	`/api/v1/hatcheries/:id/batches`	Operator — create batch
PATCH	`/api/v1/hatcheries/batches/:id/stage`	Operator — advance stage
POST	`/api/v1/hatcheries/sales`	Operator — record sale
GET	`/api/v1/hatcheries/sales/:txRef`	Buyer — QR code lookup
POST	`/api/v1/hatcheries/grow-out`	Buyer — log grow-out stocking

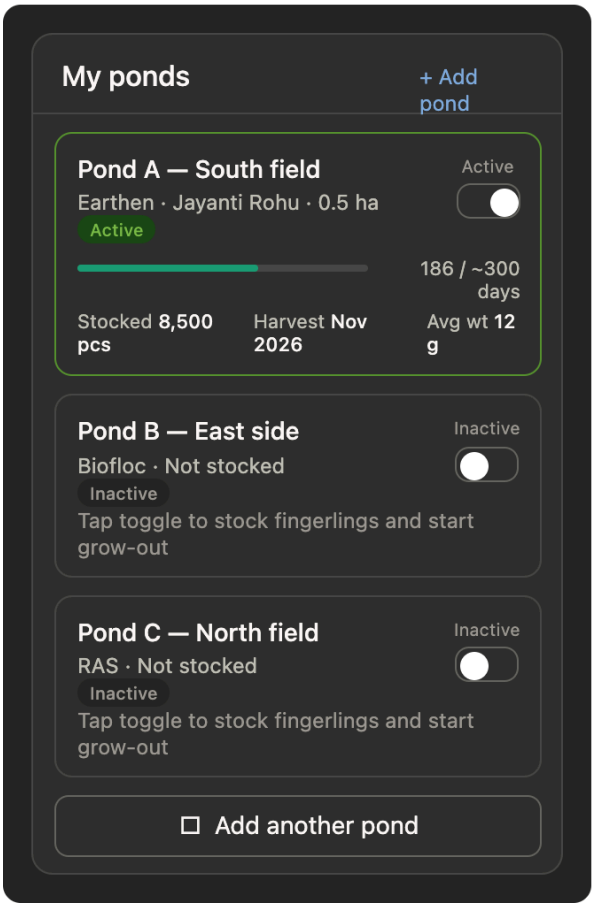
Good thinking — folding it into the existing pond feature is the right call. It avoids screen fragmentation and the pond is already the natural anchor for everything: type, species, ROI, water quality. Let me design exactly how this works.

Yes, this is exactly the right instinct — no new screen needed for the farmer side. The pond IS the container for everything. Let me first show the pond lifecycle states, then the exact UI, then the code changes.

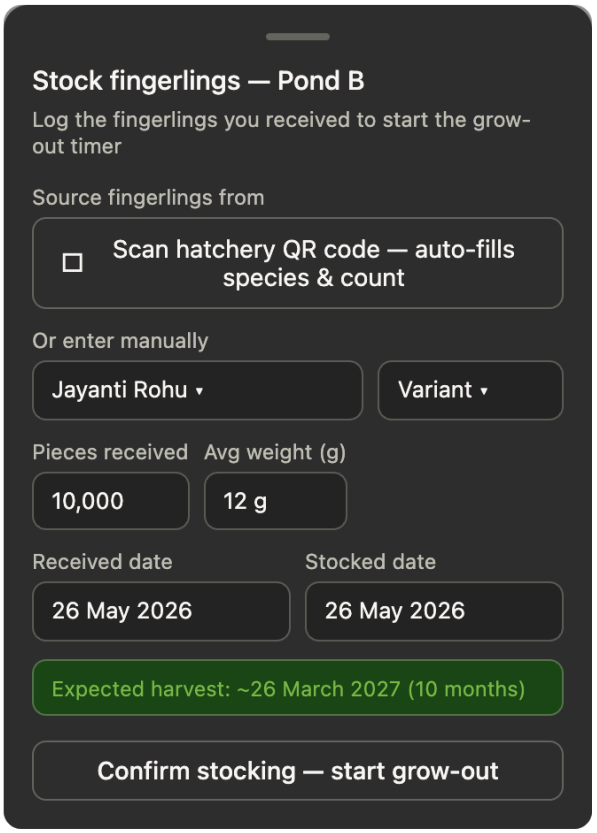
The core idea: a pond now has four states — `setup` (just created, empty), `inactive` (configured but no fish), `active` (fingerlings stocked, growing), `harvested` (cycle complete). Toggling inactive → active IS the fingerling receipt moment.

Now the interactive mockup showing how the revised `PondsList` screen looks, including the toggle and the activation sheet:

My ponds



Stock fingerlings sheet (appears on toggle)



Now the exact code changes. This is surgical — you're modifying two existing screens and adding one bottom sheet component, nothing else.

What changes and what stays the same

The existing `AddEditPond` screen keeps everything it already collects — pond type (earthen/biofloc/RAS/cage), area, location. You just remove the requirement to fill species at creation time, because species gets set at the moment of stocking. The `PondsList` screen gets the toggle and a new bottom sheet component.

Database — one migration

This is the only schema change needed for the farmer side. Add it as migration 028:

sql

```sql
-- 028_pond_lifecycle.sql

-- Add status and grow-out fields directly onto the ponds table
ALTER TABLE ponds
  ADD COLUMN IF NOT EXISTS status TEXT
    CHECK (status IN ('setup','inactive','active','harvested'))
    DEFAULT 'inactive',
  ADD COLUMN IF NOT EXISTS stocked_date DATE,
  ADD COLUMN IF NOT EXISTS fingerling_count INT,
  ADD COLUMN IF NOT EXISTS fingerling_avg_weight_g NUMERIC,
  ADD COLUMN IF NOT EXISTS fingerling_transaction_ref TEXT,   -- links to hatchery sale
  ADD COLUMN IF NOT EXISTS species_variant TEXT,              -- 'Jayanti Rohu' etc.
  ADD COLUMN IF NOT EXISTS expected_harvest_date DATE,
  ADD COLUMN IF NOT EXISTS actual_harvest_date DATE,
  ADD COLUMN IF NOT EXISTS harvest_weight_kg NUMERIC;

-- Back-fill existing active ponds so nothing breaks
UPDATE ponds SET status = 'active'
WHERE expected_harvest_date IS NOT NULL
  AND actual_harvest_date IS NULL;

UPDATE ponds SET status = 'harvested'
WHERE actual_harvest_date IS NOT NULL;
```

You're putting these fields directly on `ponds` rather than a separate table because a pond only ever has one active grow-out cycle at a time. When it gets re-stocked, you just update these fields again and the previous cycle's data lives in the `grow_out_start_logs` table for history.

WatermelonDB — add columns to existing pond model

javascript

```
// src/database/schema.js — in the existing 'ponds' tableSchema, add:
{ name: 'status',                  type: 'string' },   // 'setup'|'inactive'|'active'|'harvested'
{ name: 'stocked_date',            type: 'number', isOptional: true },
{ name: 'fingerling_count',        type: 'number', isOptional: true },
{ name: 'fingerling_avg_weight_g', type: 'number', isOptional: true },
{ name: 'fingerling_transaction_ref',type: 'string', isOptional: true },
{ name: 'species_variant',         type: 'string', isOptional: true },
{ name: 'expected_harvest_date',   type: 'number', isOptional: true },
```

Bump `version` by 1 and add the new columns to your WatermelonDB migration file.

Backend — two new endpoints on the existing ponds router

Add these to `backend/src/routes/ponds.ts` (your existing file):

typescript

```
// PATCH /api/v1/ponds/:id/stock  — farmer logs fingerling receipt → activates pond
router.patch('/:id/stock', authenticateToken, async (req, res) => {
  const {
    fingerling_count, fingerling_avg_weight_g,
    fingerling_transaction_ref, species_id, species_variant,
    stocked_date, received_date
  } = req.body;

  const stockedDt = new Date(stocked_date);
  const harvest = new Date(stockedDt);
  harvest.setMonth(harvest.getMonth() + 10);  // 10-month grow-out default

  const { rows } = await pool.query(
    `UPDATE ponds SET
       status = 'active',
       species_id = $1,
       species_variant = $2,
       fingerling_count = $3,
       fingerling_avg_weight_g = $4,
       fingerling_transaction_ref = $5,
       stocked_date = $6,
       expected_harvest_date = $7,
       updated_at = NOW()
     WHERE id = $8
     RETURNING *`,
```

```
    [species_id, species_variant, fingerling_count, fingerling_avg_weight_g,
     fingerling_transaction_ref, stocked_date,
     harvest.toISOString().split('T')[0], req.params.id]
  );

  // Also write a grow_out_start_log for hatchery traceability
  if (fingerling_transaction_ref) {
    const sale = await pool.query(
      'SELECT id FROM fingerling_sales WHERE transaction_ref = $1',
      [fingerling_transaction_ref]
    );
    if (sale.rows.length) {
      await pool.query(
        `INSERT INTO grow_out_start_logs
          (sale_id, farmer_user_id, pond_id, received_date, stocked_date,
           fingerling_count, fingerling_avg_weight_g, expected_harvest_date)
          VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
          ON CONFLICT DO NOTHING`,
        [sale.rows[0].id, (req as any).user.id, req.params.id,
          received_date, stocked_date, fingerling_count,
          fingerling_avg_weight_g, harvest.toISOString().split('T')[0]]
      );
    }
  }
  res.json(rows[0]);
});

// PATCH /api/v1/ponds/:id/harvest — farmer marks pond as harvested
router.patch('/:id/harvest', authenticateToken, async (req, res) => {
  const { harvest_weight_kg, actual_harvest_date } = req.body;
  const { rows } = await pool.query(
    `UPDATE ponds SET
       status = 'harvested',
       harvest_weight_kg = $1,
       actual_harvest_date = $2,
       updated_at = NOW()
     WHERE id = $3 RETURNING *`,
    [harvest_weight_kg, actual_harvest_date, req.params.id]
  );
  res.json(rows[0]);
});
```

Mobile — the bottom sheet component

Create this as `src/components/StockFingerlingsSheet.tsx`. It's a reusable bottom sheet that `PondsList` opens when the toggle fires:

typescript

```typescript
// src/components/StockFingerlingsSheet.tsx
import React, { useState } from 'react';
import {
  View, Text, TouchableOpacity, TextInput,
  StyleSheet, Modal, ScrollView, Alert
} from 'react-native';
import { useTheme } from '../ThemeContext';
import apiService from '../services/apiService';
import { calculateFingerlingMaturityDate } from '../services/hatcheryNotifications';

interface Props {
  visible: boolean;
  pond: any;                    // your existing pond model
  onClose: () => void;
  onStocked: (updatedPond: any) => void;
}

export default function StockFingerlingsSheet({ visible, pond, onClose, onStocked }: Props) {
  const { colors } = useTheme();
  const [txRef, setTxRef]       = useState('');
  const [species, setSpecies]   = useState('');
  const [variant, setVariant]   = useState('');
  const [count, setCount]       = useState('');
  const [avgWt, setAvgWt]       = useState('12');
  const [receivedDate, setReceivedDate] = useState(
    new Date().toISOString().split('T')[0]
  );
  const [loading, setLoading]   = useState(false);

  // Auto-fill from hatchery QR / transaction ref
  async function lookupTransaction() {
    if (!txRef.trim()) return;
    try {
      const sale = await apiService.get(`/hatcheries/sales/${txRef.trim()}`);
      setSpecies(sale.species_name);
      setVariant(sale.species_variant || '');
      setCount(String(sale.quantity_pieces || ''));
      setAvgWt(String(sale.avg_weight_g || 12));
    } catch {
      Alert.alert('Not found', 'No sale found for that reference. Enter details manually.');
    }
  }

  function getExpectedHarvest(): string {
    const dt = new Date(receivedDate);
    dt.setMonth(dt.getMonth() + 10);
    return dt.toDateString();
```

```
}

async function confirmStock() {
  if (!species || !count) {
    Alert.alert('Missing info', 'Species and fingerling count are required.');
    return;
  }
  setLoading(true);
  try {
    const updated = await apiService.patch(`/ponds/${pond.id}/stock`, {
      species_variant: variant,
      fingerling_count: parseInt(count),
      fingerling_avg_weight_g: parseFloat(avgWt),
      fingerling_transaction_ref: txRef || null,
      stocked_date: receivedDate,
      received_date: receivedDate,
    });
    onStocked(updated);
  } catch (e) {
    Alert.alert('Error', 'Could not save stocking. Please try again.');
  } finally {
    setLoading(false);
  }
}

return (
  <Modal visible={visible} animationType="slide" transparent onRequestClose={onClose}>
    <TouchableOpacity style={s.backdrop} activeOpacity={1} onPress={onClose} />
    <View style={[s.sheet, { backgroundColor: colors.surface }]}>
      <View style={s.handle} />
      <ScrollView showsVerticalScrollIndicator={false}>
        <Text style={[s.title, { color: colors.textPrimary }]}>
          Stock fingerlings — {pond?.name}
        </Text>
        <Text style={[s.sub, { color: colors.textSecondary }]}>
          Fill fingerling details to start grow-out timer
        </Text>

        {/* QR / transaction ref row */}
        <Text style={[s.label, { color: colors.textSecondary }]}>
          Hatchery transaction ref (optional)
        </Text>
        <View style={s.refRow}>
          <TextInput
            style={[s.input, { flex: 1, color: colors.textPrimary,
              borderColor: colors.surfaceAlt }]}
            value={txRef}
            onChangeText={setTxRef}
```

```
    placeholder="e.g. TXN-1748249600-ABC"
    placeholderTextColor={colors.textMuted}
    autoCorrect={false}
    autoCapitalize="characters"
  />
  <TouchableOpacity style={[s.lookupBtn, { borderColor: colors.surfaceAlt }]}
    onPress={lookupTransaction}>
    <Text style={{ color: colors.primary, fontSize: 13 }}>Look up</Text>
  </TouchableOpacity>
</View>

{/* Species + variant */}
<Text style={[s.label, { color: colors.textSecondary }]}>Species</Text>
<TextInput
  style={[s.input, { color: colors.textPrimary, borderColor: colors.surfaceAlt }]}
  value={species}
  onChangeText={setSpecies}
  placeholder="e.g. Rohu, Catla, Mrigal"
  placeholderTextColor={colors.textMuted}
/>
<Text style={[s.label, { color: colors.textSecondary }]}>Variant (if enhanced)</Text>
<TextInput
  style={[s.input, { color: colors.textPrimary, borderColor: colors.surfaceAlt }]}
  value={variant}
  onChangeText={setVariant}
  placeholder="e.g. Jayanti Rohu, Amrita Katla"
  placeholderTextColor={colors.textMuted}
/>

{/* Count + weight */}
<View style={{ flexDirection: 'row', gap: 10 }}>
  <View style={{ flex: 1 }}>
    <Text style={[s.label, { color: colors.textSecondary }]}>Pieces received</Text>
    <TextInput
      style={[s.input, { color: colors.textPrimary, borderColor: colors.surfaceAlt }]}
      value={count} onChangeText={setCount}
      keyboardType="numeric" placeholder="e.g. 10000"
      placeholderTextColor={colors.textMuted}
    />
  </View>
  <View style={{ flex: 1 }}>
    <Text style={[s.label, { color: colors.textSecondary }]}>Avg weight (g)</Text>
    <TextInput
      style={[s.input, { color: colors.textPrimary, borderColor: colors.surfaceAlt }]}
      value={avgWt} onChangeText={setAvgWt}
      keyboardType="decimal-pad" placeholder="12"
      placeholderTextColor={colors.textMuted}
    />
```

```
            </View>
          </View>

          {/* Date */}
          <Text style={[s.label, { color: colors.textSecondary }]}>
            Date received &amp;amp; stocked
          </Text>
          <TextInput
            style={[s.input, { color: colors.textPrimary, borderColor: colors.surfaceAlt }]}
            value={receivedDate} onChangeText={setReceivedDate}
            placeholder="YYYY-MM-DD"
            placeholderTextColor={colors.textMuted}
            autoCorrect={false}
          />

          {/* Harvest preview */}
          {count ? (
            <View style={[s.harvestBox, { backgroundColor: colors.surfaceAlt }]}>
              <Text style={{ color: colors.textSecondary, fontSize: 12 }}>
                Expected harvest date
              </Text>
              <Text style={{ color: colors.primary, fontSize: 14, fontWeight: '500' }}>
                {getExpectedHarvest()} (~10 months)
              </Text>
            </View>
          ) : null}

          <TouchableOpacity
            style={[s.confirmBtn, { opacity: loading ? 0.6 : 1 }]}
            onPress={confirmStock}
            disabled={loading}
          >
            <Text style={s.confirmText}>
              {loading ? 'Saving...' : 'Confirm stocking — start grow-out'}
            </Text>
          </TouchableOpacity>
        </ScrollView>
      </View>
    </Modal>
  );
}

const s = StyleSheet.create({
  backdrop:   { flex: 1, backgroundColor: 'rgba(0,0,0,0.4)' },
  sheet:      { borderTopLeftRadius: 16, borderTopRightRadius: 16,
                padding: 20, paddingBottom: 40, maxHeight: '90%' },
  handle:     { width: 36, height: 4, borderRadius: 2,
                backgroundColor: '#ccc', alignSelf: 'center', marginBottom: 16 },
```

```
  title:      { fontSize: 16, fontWeight: '500', marginBottom: 4 },
  sub:        { fontSize: 13, marginBottom: 16 },
  label:      { fontSize: 12, marginBottom: 4, marginTop: 8 },
  input:      { borderWidth: 0.5, borderRadius: 8, padding: 10,
                fontSize: 14, marginBottom: 4 },
  refRow:     { flexDirection: 'row', gap: 8, marginBottom: 4 },
  lookupBtn:  { borderWidth: 0.5, borderRadius: 8, paddingHorizontal: 12,
                justifyContent: 'center' },
  harvestBox: { borderRadius: 8, padding: 10, marginTop: 8, marginBottom: 8 },
  confirmBtn: { backgroundColor: '#1D9E75', borderRadius: 10,
                padding: 14, alignItems: 'center', marginTop: 12 },
  confirmText:{ color: '#fff', fontSize: 15, fontWeight: '500' },
});
```

PondsList screen — the key changes

In your existing `PondsList` screen, these are the three things you change:

typescript

```
// 1. State: track which pond's sheet is open
const [stockingPond, setStockingPond] = useState<any>(null);

// 2. Toggle handler — fires when farmer taps the switch
function handleToggle(pond: any) {
  if (pond.status === 'inactive' || pond.status === 'setup') {
    setStockingPond(pond);          // opens the StockFingerlingsSheet
  } else if (pond.status === 'active') {
    // Confirm before deactivating
    Alert.alert(
      'Mark as harvested?',
      'This will end the current grow-out cycle and let you log harvest weight.',
      [
        { text: 'Cancel', style: 'cancel' },
        { text: 'Yes, harvest', onPress: () => navigateTo('HarvestLog', { pond }) }
      ]
    );
  }
}

// 3. Pond card — add these two UI elements to each existing card:
// (a) The active/inactive badge next to species name
// (b) The Switch component

<Switch
  value={pond.status === 'active'}
  onValueChange={() => handleToggle(pond)}
  trackColor={{ false: colors.surfaceAlt, true: colors.primary }}
```

```
  thumbColor="#fff"
/>

// 4. Render the sheet at the bottom of the screen
<StockFingerlingsSheet
  visible={!!stockingPond}
  pond={stockingPond}
  onClose={() => setStockingPond(null)}
  onStocked={(updated) => {
    // Update local pond list state
    setPonds(prev => prev.map(p => p.id === updated.id ? updated : p));
    setStockingPond(null);
  }}
/>
```

How the active pond card changes (display only)

When `pond.status === 'active'`, the card shows everything it already shows now (pond type, area, ROI estimate) plus three new lines that come from the new fields:

typescript

```
// Add these inside the existing active pond card render:
{pond.status === 'active' && pond.stocked_date && (
  <>
    <Text>{pond.species_variant || pond.species_name} ·
{pond.fingerling_count?.toLocaleString()} pcs</Text>
    <ProgressBar
      progress={daysSince(pond.stocked_date) / 300}   // 300-day grow-out
      color={colors.primary}
    />
    <Text>
      Day {daysSince(pond.stocked_date)} of grow-out ·
      Harvest ~{formatDate(pond.expected_harvest_date)}
    </Text>
  </>
)}
```

The `Harvest Countdown` strip on your Home dashboard already reads from `expected_harvest_date` on each pond — so as soon as `confirmStock()` saves the field, that pond card automatically appears in the Home countdown strip with no extra work.

The harvest flow (toggle OFF)

When the farmer toggles an active pond off, navigate them to a lightweight `HarvestLog` screen (which you can make a modal/bottom sheet too, same pattern as above):

typescript

```
// Minimal harvest log — update the pond and write final ROI
await apiService.patch(`/ponds/${pond.id}/harvest`, {
  harvest_weight_kg: harvestKg,
  actual_harvest_date: today,
});
// Navigate to EconomicsResult with pre-filled actuals for the ROI comparison
navigation.navigate('EconomicsResult', {
  pond,
  actualYieldKg: harvestKg,
  cycleStartDate: pond.stocked_date,
  cycleEndDate: today,
});
```

That last line closes the loop — the farmer's actual harvest weight flows straight into the existing Economics/ROI screen, so they see projected vs actual BCR for that grow-out cycle.

The summary of what you're touching: one migration, two new API endpoints on an existing router, one new `StockFingerlingsSheet` component, and about 40 lines added to `PondsList`. The `AddEditPond` screen barely changes — you just make the species field optional since it gets filled at stocking time instead.